Laboratorio de Control

Práctica 3: Simulink y Control con MATLAB

Programa de Ingeniería Electrónica
Facultad de Ingeniería – Universidad de Antioquia
2026-1

Profesores: *Hernan Felipe Garcia, Andres Felipe Lopez*
Alumno: *Juan Pablo González Blandón*
Fecha: *17 Marzo 2026*

Índice

1. Resultados de aprendizaje **3**

2. Preinforme **3**
2.1. Descripción de comandos de MATLAB 3
2.2. Descripción de bloques de Simulink 4

3. Procedimiento **5**
3.1. Numeral 4.1 – Exploración del Library Browser 5
3.2. Numeral 4.2 – Montaje del Ejemplo 1 5
3.3. Numeral 4.3 – Scope y bloque Mux 6
3.4. Numeral 4.4 – Diferencia entre Out y To Workspace 7
3.5. Numeral 4.5 – Parametrización y primera simulación 7
3.6. Numeral 4.6 – Diferencia entre lsim y sim 9
3.7. Numeral 4.7 – Ejecución de ejemplo1 desde la VCML 9
3.8. Numeral 4.8 – Ejemplo 2 con saturación 10

4. Conclusiones **14**

5. Bibliografía **15**

1. Resultados de aprendizaje

1.1. Conocer, con base en ejemplos, los procedimientos más comunes de la simulación de sistemas de control con el uso de **Simulink** (SL), la herramienta gráfica de MATLAB.

1.2. Aprender la forma de conectar los espacios de trabajo de MATLAB (ML) y Simulink (SL).

1.3. Obtener en SL la respuesta transitoria de sistemas de control lineales y no lineales.

2. Preinforme

2.1. Descripción de comandos de MATLAB

`lsim`

Simula la respuesta en el tiempo de un sistema dinámico lineal e invariante (LTI) a una entrada arbitraria. Requiere un modelo definido en MATLAB mediante objetos `tf`, `ss` o `zpk`, junto con un vector de tiempo `t` y un vector de entrada `u`. **No** puede ejecutar modelos Simulink `.slx`.

`sim`

Ejecuta una simulación de un modelo Simulink (`.slx`) desde la línea de comandos de MATLAB. A diferencia de `lsim`, es capaz de simular sistemas no lineales, discretos e híbridos. Los resultados se encapsulan en un objeto `Simulink.SimulationOutput`. Ejemplo de uso:

```
out = sim('ejemplo1', 'StopTime', '5');
```

`linmod`

Extrae el modelo lineal en espacio de estados de un diagrama Simulink, linearizando el sistema alrededor de un punto de operación (por defecto $x = 0$, $u = 0$). Devuelve las matrices A, B, C, D de la representación:

$$\dot{x} = Ax + Bu, \qquad y = Cx + Du \tag{1}$$

`trim`

Determina el punto de equilibrio (punto de *trim*) de un sistema dinámico, es decir, los valores de estados y entradas para los cuales las derivadas del estado son iguales a cero. Utiliza un algoritmo de programación cuadrática secuencial partiendo de un punto inicial dado.

`simset`

Función declarada obsoleta (*deprecated*) desde MATLAB R2009b. Anteriormente se usaba para crear estructuras de opciones de simulación (solver, tolerancias, workspace) que se pasaban al comando `sim`. Su reemplazo moderno es el objeto `Simulink.SimulationInput`.

`simget`

Complemento de `simset`: leía los parámetros de configuración actuales de un modelo Simulink desde la línea de comandos. También fue declarado obsoleto junto con `simset` en versiones posteriores a R2009b.

2.2. Descripción de bloques de Simulink

`Sign`

Implementa la función signo matemática: devuelve $+1$ si la entrada es positiva, -1 si es negativa y 0 si es cero. Introduce no linealidad en el sistema; útil para modelar fricción de Coulomb o control tipo relé.

`In` **(Inport)**

Puerto de entrada del modelo. Permite recibir datos desde MATLAB cuando el modelo se ejecuta con el comando `sim`, pasando señales externas como entrada al diagrama. Posee tres pestañas de configuración: *Main* (número de puerto), *Signal Attributes* (tipo de dato, dimensiones, rango) y *Execution* (tiempo de muestreo, interpolación de datos y manejo de eventos).

`Out` **(Outport)**

Puerto de salida del modelo. Envía la señal de salida al workspace de MATLAB cuando la simulación se ejecuta con `sim`. Posee las mismas tres pestañas que `In`, con la diferencia de que **no** incluye la opción de interpolación de datos ni la tabla de eventos en la pestaña *Execution*.

`To Workspace`

Guarda cualquier señal interna del diagrama directamente en el workspace de MATLAB con un nombre configurable (por defecto `simout`). A diferencia del puerto `Out`, funciona independientemente del método de simulación (desde SL o desde `sim`). Sus parámetros incluyen: nombre de variable, límite de puntos, decimación, formato de guardado (*Timeseries*, *Array*, *Structure*) y tiempo de muestreo.

`Transfer Fcn`

Implementa una función de transferencia continua:

$$H(s) = \frac{N(s)}{D(s)} \tag{2}$$

Los coeficientes del numerador y denominador se ingresan como vectores de potencias descendentes de s.

3. Procedimiento

3.1. Numeral 4.1 – Exploración del Library Browser

Para abrir Simulink se ejecuta en la ventana de comandos de MATLAB:

```
>> simulink
```

Las sublibrerías encontradas en cada librería principal son:

Cuadro 1: Sublibrerías del Library Browser

Librería	Sublibrerías
Simulink	Commonly Used Blocks, Continuous, Dashboard, Discontinuities, Discrete, Logic and Bit Operations, Lookup Tables, Math Operations, Matrix Operations, Messages & Events, Model Verification, Model-Wide Utilities, Ports & Subsystems, Signal Attributes, Signal Routing, Sinks, Sources, String, User-Defined Functions, Additional Math & Discrete, Quick Insert
Control System Toolbox	Linear Parameter Varying, State Estimation
Simulink Extras	Additional Discrete, Additional Linear, Additional Lookup Tables, Additional Sinks, Flip Flops, FMU Import, Linearization, Transformations

3.2. Numeral 4.2 – Montaje del Ejemplo 1

Se montó el sistema de control retroalimentado (SCR) de la Figura 1, compuesto por un controlador PD real (compensador de adelanto) y un proceso de doble integrador $\frac{1}{s^2}$.

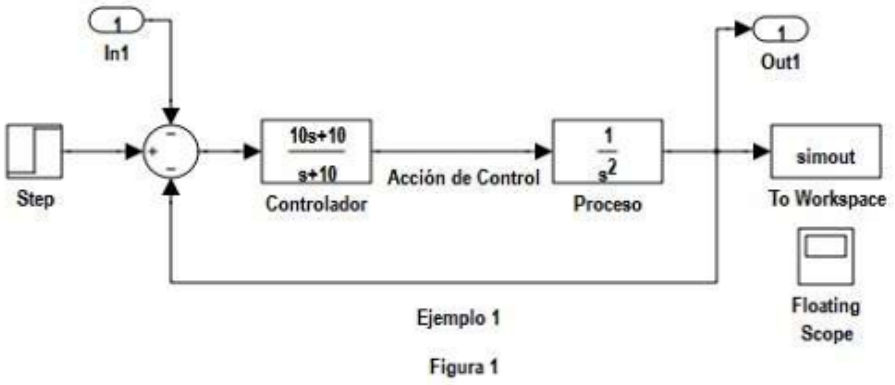

Figura 1: Diagrama de bloques – Ejemplo 1 (ejemplo1.slx)

Los parámetros del controlador son:

$$C(s) = \frac{10s + 10}{s + 10} \tag{3}$$

Y el proceso:

$$P(s) = \frac{1}{s^2} \tag{4}$$

3.3. Numeral 4.3 – Scope y bloque Mux

Se exploró el *Floating Scope* y sus opciones de configuración, en particular la pestaña *Logging* que permite enviar los datos de la gráfica al workspace de MATLAB en formatos *structure with time*, *structure* o *array*.

Se agregó un bloque `Mux` con tres entradas conectadas a las señales de interés del sistema:

1. Señal de entrada (salida del Step / In1)
2. Acción de control (salida del controlador)
3. Señal de salida (salida del proceso)

La salida del Mux se conectó a un bloque `Scope` normal para visualizar las tres señales simultáneamente.

3.4. Numeral 4.4 – Diferencia entre Out y To Workspace

Cuadro 2: Comparación entre bloques Out (Outport) y To Workspace

Característica	`Out` (Outport)	`To Workspace`
¿Cuándo guarda?	Solo al usar `sim` desde MATLAB	Siempre, sin importar cómo se corra
Variable generada	Campo `yout` dentro del objeto `SimulationOutput`	Variable independiente en el workspace (nombre configurable)
Pestañas de config.	Main, Signal Attributes, Execution (sin interpolación ni eventos)	Solo Parameters
Parámetros clave	Número de puerto, tipo de dato, dimensiones	Nombre, límite de puntos, decimación, formato de guardado
Uso típico	Interfaz MATLAB ↔ Simulink en scripts	Registro y depuración de señales internas

Las formas disponibles de enviar información de la simulación al workspace son:

1. Bloque `To Workspace`
2. Puerto `Out` combinado con el comando `sim`
3. Bloque `Scope` con la opción *Logging* activada

3.5. Numeral 4.5 – Parametrización y primera simulación

Se configuró el entorno de simulación en *Model Settings*:

- **Solver:** tipo variable-step, método `auto`, tiempo de inicio 0s, tiempo de parada 10s.
- **Data Import/Export:** se activó el guardado de `time` (tout), `states` (xout) y `output` (yout) en formato *Array*.

Tras ejecutar la simulación con el botón *Run*, las variables generadas en el workspace fueron:

Cuadro 3: Variables generadas en el workspace tras la simulación

Variable	Tamaño	Tipo	Origen
`tout`	57×1	double	Data Import/Export – Time
`xout`	57×3	double	Data Import/Export – States
`yout`	57×1	double	Data Import/Export – Output
`simout`	1×1	timeseries	Bloque To Workspace

La variable `simout` es una variable **no esperada** en el sentido de que no proviene de la configuración del solver, sino del bloque `To Workspace` que actúa de forma independiente.

La matriz `xout` tiene 3 columnas porque el sistema posee 3 variables de estado: una del controlador (Transfer Fcn de orden 1) y dos de los integradores. Para extraer una columna de una matriz se usa indexación:

```
estado1 = xout(:, 1);    % primera columna completa
estado2 = xout(:, 2);
estado3 = xout(:, 3);
```

La respuesta del sistema se graficó con:

```
plot(tout, yout)
grid on
xlabel('Tiempo (s)')
ylabel('Amplitud')
title('Respuesta del sistema - Ejemplo 1')
```

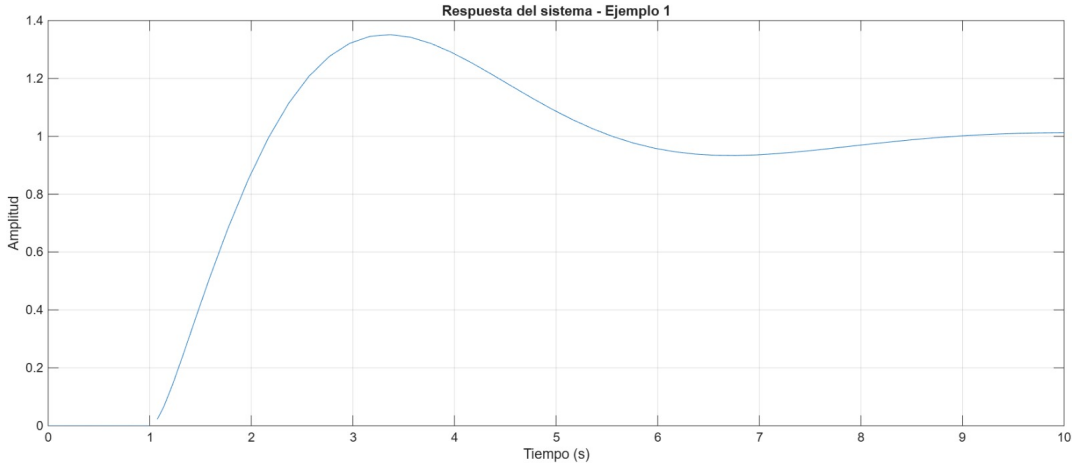

Figura 2: Respuesta temporal del sistema de control – Ejemplo 1

Se observa que el sistema presenta un sobrenivel aproximado del 34 % y se estabiliza en el valor unitario hacia $t \approx 9\,\mathrm{s}$, comportamiento típico de un sistema de segundo orden subamortiguado.

3.6. Numeral 4.6 – Diferencia entre lsim y sim

¿Qué diferencia importante se da entre `lsim` y `sim`?

Cuadro 4: Comparación entre `lsim` y `sim`

Característica	`lsim`	`sim`
¿Qué ejecuta?	Modelos LTI de MATLAB (`tf`, `ss`, `zpk`)	Modelos Simulink (`.slx`)
Sistemas no lineales	No	Sí
Requiere Simulink	No	Sí
Entrada	Vector `u` directo	Puerto `In` del modelo
Salida	Variable directa `y`	Objeto `SimulationOutput`

¿Por qué son importantes los iconos In port y Out port?

Los puertos `In` y `Out` son la interfaz entre Simulink y MATLAB. Permiten que un script de MATLAB controle la simulación programáticamente: el puerto `In` recibe la señal de entrada definida en MATLAB, y el puerto `Out` retorna la señal de salida al objeto `SimulationOutput`. Sin ellos, el modelo solo puede ejecutarse manualmente desde Simulink.

3.7. Numeral 4.7 – Ejecución de ejemplo1 desde la VCML

Se ejecutaron los siguientes comandos en la ventana de comandos:

```
>> whos                                          % verificar variables actuales
>> clear tout xout yout simout                   % borrar variables previas
>> whos                                          % verificar workspace limpio
>> out = sim('ejemplo1', 'StopTime', '5')        % correr modelo
>> whos                                          % verificar nuevas variables
```

¿Cuántas variables tiene el ETML? Una sola: `out`.

¿De qué tipo? `Simulink.SimulationOutput` – un objeto que encapsula todos los resultados:

```
out =
  Simulink.SimulationOutput:
    simout: [1x1 timeseries]
    tout:   [53x1 double]
    xout:   [53x3 double]
    yout:   [53x1 double]
    SimulationMetadata: [1x1 Simulink.SimulationMetadata]
```

```
    ErrorMessage: [0x0 char]
```

¿Tienen alguna relación? Sí – todas son campos del mismo objeto `out`. Esta es la diferencia clave con la simulación directa desde SL, donde cada variable queda suelta en el workspace.

¿Cómo cambia si se ejecuta `out = sim('ejemplo1')`? Sin especificar `StopTime`, MATLAB usa la sintaxis antigua y el resultado cambia completamente: aparecen **dos variables** separadas – `out` como vector `double` (57×1) y `simout` como `timeseries` independiente. El encapsulamiento en el objeto `SimulationOutput` desaparece.

3.8. Numeral 4.8 – Ejemplo 2 con saturación

Montaje del modelo

Se construyó el modelo `ejemplo2.slx` agregando un bloque `Saturation` con límites $[-1, +1]$ entre el nodo de suma y el compensador, según la Figura 3.

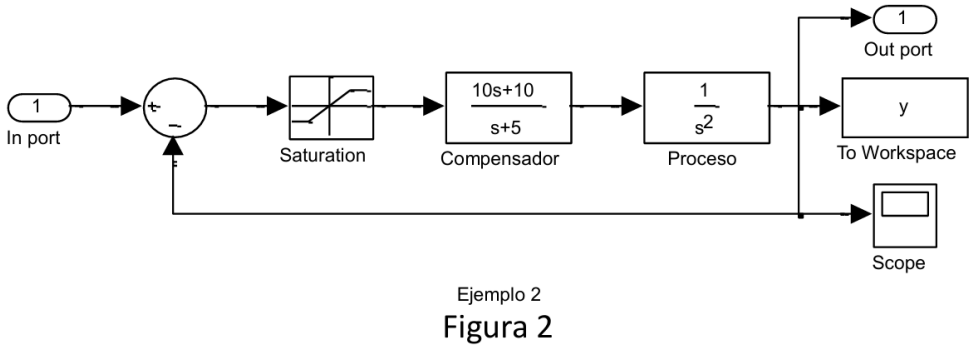

Figura 3: Diagrama de bloques – Ejemplo 2 con saturación (ejemplo2.slx)

Los parámetros del sistema son:

$$C(s) = \frac{10s + 10}{s + 5}, \qquad P(s) = \frac{1}{s^2} \tag{5}$$

Efecto de la saturación

La saturación está ubicada **antes del controlador**, limitando la señal de error entre -1 y $+1$. Esto significa que, sin importar cuán grande sea la referencia u, el controlador nunca recibe una señal de error mayor que 1.

Para entender su efecto, se graficó el error real versus el error saturado para $u = 4$ (Figura 4):

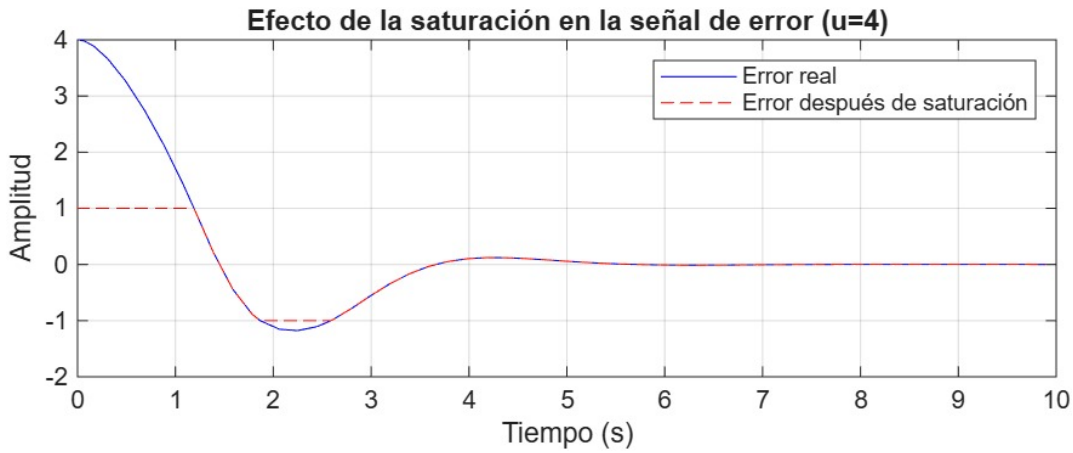

Figura 4: Efecto de la saturación en la señal de error ($u = 4$)

Se identifican tres zonas:

1. $t \in [0, 1{,}5]$s – **saturación activa:** el error real vale 4 pero el controlador solo recibe 1. La saturación "miente" al controlador, reduciéndole la señal de mando.
2. $t \approx 1{,}5$s – **fin de saturación:** el error real baja por debajo de 1 y las dos curvas se unen. El sistema opera libremente.
3. $t \in [2, 2{,}5]$s – **saturación negativa breve:** la salida supera la referencia (sobrenivel) y el error se vuelve negativo, activando brevemente el límite inferior -1.

Script ejemplo2m.m

```
% ejemplo2m.m
% Respuesta temporal del sistema ejemplo2 para entradas u = 1 a 7

t = (0:0.1:9.9)';

figure
hold on

for i = 1:7
    assignin('base', 't_ext', t);
    assignin('base', 'u_ext', i*ones(size(t)));

    out = sim('ejemplo2', 'StopTime', '9.9', ...
              'LoadExternalInput', 'on', ...
              'ExternalInput', '[t_ext, u_ext]');

    tt   = out.tout;
    yy   = out.yout;
    yint = interp1(tt, yy, t, 'linear', 'extrap');
    y0(1:length(yint), i) = yint;
end

```

```
sy   = size(y0);
y0ss = y0(sy(1), :);
SP   = 100 * (max(y0) - y0ss) ./ y0ss;

plot(t, y0); grid on
xlabel('Tiempo (s)')
ylabel('Amplitud')
title('Respuesta con saturacion -- entradas 1 a 7')
legend(arrayfun(@(x) sprintf('u=%d', x), 1:7, 'UniformOutput', false))
```

Resultados

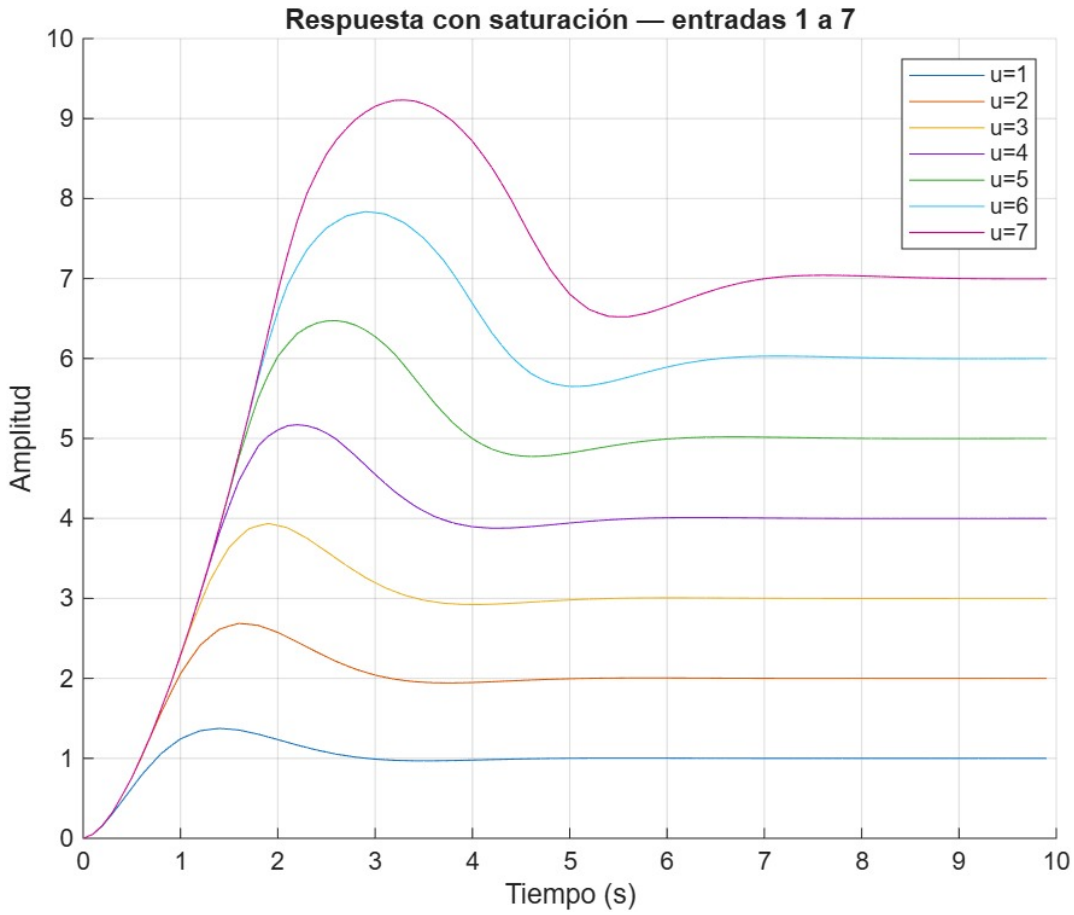

Figura 5: Respuestas temporales del sistema con saturación para $u = 1, 2, \ldots, 7$

Cuadro 5: Sobrenivel porcentual por valor de entrada

Entrada u	SP (%)
1	37.44
2	34.34
3	31.21
4	29.29
5	29.48
6	30.63
7	31.94

¿Cuántas respuestas tiene la variable generada por el Scope? Una por cada corrida del bucle `for`, es decir 7 respuestas, ya que el Scope registra la señal en cada ejecución del modelo.

¿Cuántas respuestas tiene la variable To Workspace? También 7, una por cada iteración del bucle.

¿Cuántas respuestas tiene y0? 7 columnas, cada una correspondiente a una entrada $u = 1, 2, \ldots, 7$.

¿Cuál es la respuesta con mayor sobrenivel porcentual? $u = 1$ con SP $= 37{,}44\,\%$. Con entradas pequeñas el error inicial no supera el límite de saturación, por lo que el sistema se comporta linealmente y el controlador actúa con plena autoridad, generando mayor sobrenivel.

¿Cuál es la respuesta con menor tiempo de estabilización? $u = 1$, que al tener menor amplitud converge más rápidamente al valor final.

Comando linmod y verificación manual

Se ejecutó el comando `linmod` sobre el modelo:

```
>> [A, B, C, D] = linmod('ejemplo2')
```

Obteniendo las matrices:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ -10 & 0 & -40 \\ -1 & 0 & -5 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 10 \\ 1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \end{bmatrix} \tag{6}$$

Verificación manual. El sistema tiene 3 variables de estado: x_1 (estado del compensador), x_2 (salida del primer integrador) y x_3 (salida del segundo integrador $= y$). Ignorando la saturación (ganancia unitaria en el punto de operación), las ecuaciones de estado se derivan así:

Del compensador $C(s) = \frac{10s+10}{s+5}$, definiendo x_1 como estado interno:

$$\dot{x}_1 = -5x_1 + e \tag{7}$$
$$u_c = -40x_1 + 10e \tag{8}$$

De los integradores:

$$\dot{x}_2 = u_c = -40x_1 + 10e \tag{9}$$
$$\dot{x}_3 = x_2 \tag{10}$$

Sustituyendo $e = u - x_3$ (retroalimentación negativa):

$$\dot{x}_1 = -5x_1 - x_3 + u \tag{11}$$
$$\dot{x}_2 = -40x_1 - 10x_3 + 10u \tag{12}$$
$$\dot{x}_3 = x_2 \tag{13}$$

En forma matricial con orden de estados $[x_1, x_2, x_3]$:

$$A_{\text{manual}} = \begin{bmatrix} -5 & 0 & -1 \\ -40 & 0 & -10 \\ 0 & 1 & 0 \end{bmatrix}, \quad B_{\text{manual}} = \begin{bmatrix} 1 \\ 10 \\ 0 \end{bmatrix}, \quad C_{\text{manual}} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \tag{14}$$

Las matrices de MATLAB y las derivadas manualmente son equivalentes – describen el mismo sistema con una permutación del orden de los estados, dado que MATLAB asigna x_1 al estado ubicado más a la izquierda en el diagrama de bloques.

4. Conclusiones

1. **Simulink como herramienta de simulación.** Simulink permite modelar y simular sistemas dinámicos de forma gráfica, conectando bloques que representan componentes matemáticos. Esto facilita la comprensión visual de la dinámica del sistema sin necesidad de resolver analíticamente las ecuaciones diferenciales, y permite extender fácilmente el modelo hacia configuraciones no lineales.

2. **Conexión MATLAB–Simulink.** Existen múltiples formas de intercambiar información entre los dos entornos. El uso de `sim` con la sintaxis moderna encapsula todos los resultados en un objeto `Simulink.SimulationOutput`, lo que facilita el manejo programático de los datos. El bloque `To Workspace` opera de forma independiente y es más conveniente para registrar señales internas durante el desarrollo.

3. **Efecto de la saturación.** La saturación ubicada antes del controlador limita la señal de error entre -1 y $+1$. Esto produce que entradas pequeñas ($u = 1$) generen mayor sobrenivel porcentual (SP $= 37{,}44\,\%$) porque la saturación no se activa y el sistema

opera linealmente. Para entradas grandes la saturación recorta el error durante la subida inicial, reduciendo la energía acumulada y por tanto el sobrenivel relativo. Este es un fenómeno no lineal: en un sistema lineal el SP sería constante independientemente de la magnitud de la entrada.

4. **Linealización con `linmod`.** El comando `linmod` extrajo automáticamente las matrices A, B, C, D del sistema no lineal linealizado. La verificación manual confirmó la consistencia del resultado, con la única diferencia de un reordenamiento de las variables de estado respecto al convenio interno de MATLAB, que asigna x_1 al integrador ubicado más a la izquierda en el diagrama.

5. **Diferencia entre `lsim` y `sim`.** `lsim` es suficiente para sistemas lineales definidos en MATLAB, pero `sim` es indispensable cuando el sistema contiene no linealidades como la saturación. Esta distinción es fundamental en el diseño de controladores reales, donde los actuadores siempre tienen límites físicos que introducen no linealidades.

5. Bibliografía

1. MathWorks, "Simulink", *Help Document in MATLAB releases 2013, 2016, 2020, 2022*. Disponible en: https://www.mathworks.com/help/simulink/

2. Shashian, B. and Hassul, M., *Control Systems Design Using MATLAB*, Prentice Hall, 1993, p. 154.

3. MathWorks, "`sim` – Simulate Simulink model", https://www.mathworks.com/help/simulink/slref/sim.html

4. MathWorks, "`linmod` – Extract linear state-space model around operating point", https://www.mathworks.com/help/simulink/slref/linmod.html